

40 RAG Interview Questions

From basic concepts to production debugging.

Not just definitions.

SYSTEMS COVERED:

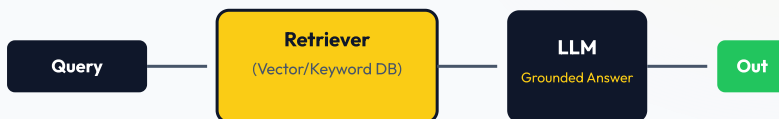
Parsing — Chunking — Embeddings — Retrieval — Reranking — Context — Gen — Eval

40 Q&As — Architectures — Scenarios →

What is **RAG** ?

Retrieval-Augmented Generation interfaces LLMs with database retrieval networks.

Instead of answering strictly using its internal weights, the system first retrieves external documents matching the query to construct prompt contexts.



CORE CONCEPT

Retrieve relevant knowledge at inference time instead of trying to encode everything in the model parameters.

What are the **main components** ?

A production RAG pipeline consists of two major operational phases:



ENGINEERING TIP

Optimizing the LLM prompt without fixing the indexing, parsing, and retrieval layers is a common design error.

Indexing vs Retrieval

These two phases happen at different times in the lifecycle:

- **Indexing** Prepares the documents, cuts them into chunks, gets embedding **(Offline):** vectors, and indexes them in a database.
- **Retrieval** Triggered by a user query. Computes the query embedding, **(Online):** searches for closest matches, and builds the context.

INTERVIEW INSIGHT

If retrieval quality is poor, replacing the LLM will not fix your system. Garbage retrieval → poor context → poor answer.

Why not send the whole file ?

Sending entire massive documents into the context window triggers several operational issues:



TAKEAWAY

RAG is not about giving the model more data; it's about supplying the right details at the right time.

Why is **chunking** necessary?

Embedding a large, multi-topic document as a single vector makes retrieval coarse and noisy.



REASON

Chunking breaks documents into topic-focused blocks, ensuring vector similarity matches specific details rather than document averages.

What is **fixed-size chunking**?

Fixed-size chunking splits text based strictly on predefined token or character size counts, regardless of document syntax layout.



LIMITATION

While simple to write, fixed chunking can truncate logical lists, tables, or sentences right in the middle.

What is **semantic chunking**?

Instead of splitting solely by token count, semantic chunking identifies document structural syntax boundaries:

- Chapter or section headings
- Markdown code blocks or lists
- Paragraph and sentence terminations
- Shift in embedding similarity between sentences

WHY IT HELPS

Chunks representing a single coherent idea are easier to retrieve and provide cleaner context for the LLM to interpret.

How to choose chunk size ?

There is no universal optimal chunk size. Structure sizing depends entirely on corpus document layout:

 **Legal text:** Requires larger chunks to preserve complete clauses.

 **Code:** Split at class/function block boundaries.

 **FAQs:** Keep questions and answers unified as single chunks.

DESIGN RULE

Do not copy tutorial settings. Benchmark different sizes (e.g. 256 vs 512 tokens) against your actual target query distributions.



Why use chunk **overlap**?

Overlap repeats key transition tokens at adjacent boundaries to maintain semantic continuity.

Without overlap, a sentence split across two chunks (e.g. "Employees can claim travel expenses if..." / "...the trip was approved by their manager") loses context.

- + **Pros:** Preserves sentence boundaries context.
- **Cons:** Uses more database storage and prompt token count.

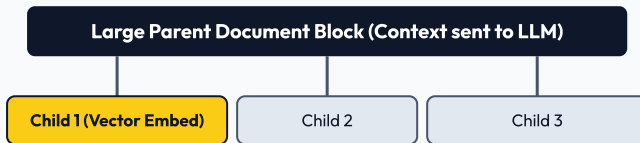
RULE OF THUMB

Starting with a 10% to 20% overlap (e.g., 50 tokens overlap for 256-sized chunks) is standard practice for continuous prose.



What is **parent-child** chunking?

It separates the small text unit used for vector database search from the larger text context supplied to the LLM.



BENEFIT

Child chunks optimize search index match accuracy. Parent blocks provide necessary logical context to the reasoning engine.

How should you chunk **tables**?

Converting a table into ordinary text strings destroys row and column coordinate relationships, causing model confusion.

Example: "Product Price A 100 B 120" is ambiguous.

Flat Text parsing (Bad)

"Product Price A 100 B 120"

Table-Aware Schema (Good)

```
[{"row": 1, "col": "A"}]
```

Preserve coordinate context explicitly

SOLUTION

Use structural parsers that preserve coordinates explicitly, such as Markdown table layouts or JSON objects.



Handling PDF layout parsing

PDF is a formatting layout, not structured database code. To prepare PDFs for chunking:

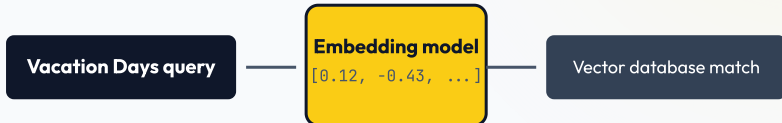
- ✓ Filter out header, footer, and page count noise.
- ✓ Reconstruct multi-column text correctly.
- ✓ OCR scanned images to capture graphical text.
- ✓ Attach metadata containing section headers.

LESSON

Poor PDF extraction guarantees broken search relevance before your chunking algorithm even runs.

Role of **embeddings** in RAG

Embeddings translate text queries and database documents into matching high-dimensional vector spaces.



CRITICAL PATH

The embedding model is the foundation of retrieval quality. A weak model returns irrelevant chunks.

What is **vector similarity search**?

Comparing high-dimensional document vectors against query vectors using mathematical distance metrics:

- **Cosine Similarity:** Measures vector angles, ignoring vector magnitude scales.
- **Dot Product:** Measures angle and length; highly optimized for unit vectors.
- **Euclidean Distance:** Calculates straight-line distance.

PURPOSE

Vector databases run these metrics to quickly return nearest neighbor candidates in the database coordinate index.

What is ANN search?

Approximate Nearest Neighbor (ANN) search trades a tiny accuracy ratio for massive indexing speeds.

Instead of matching query vectors against every single database record, ANN groups vectors using navigation graphs or trees.

 **HNSW:** Navigation graphs that traverse layer nodes.

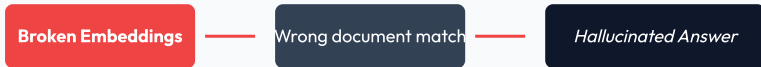
 **IVF:** Buckets search spaces into localized clusters.

INTERVIEW INSIGHT

We choose speed and scalability over mathematical precision. Real-world RAG demands low query-time latencies.

Broken embedding **impact**

A weak embedding model compromises your entire retrieval layer:



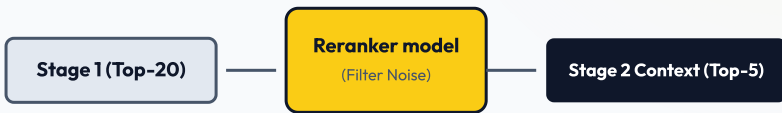
DEBUGGING FOCUS

When RAG output fails, isolate and test the vector similarity scores before changing LLM prompt instructions.

What is **Top-K retrieval**?

Top-K defines the number of candidate text chunks retrieved from vector search databases.

However, passing raw Top-K results directly to the prompt is risky. A more robust workflow includes a validation step:



BENEFIT

The retriever targets high recall rates; the reranker optimizes precise document selection for the prompt.

Why can high Top-K hurt RAG?

Adding too many chunks to query contexts degrades response quality in several ways:

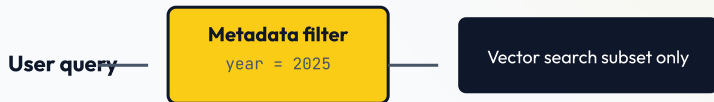
- **Token Bloat:** Exponentially drives up input API costs.
- **Attention dilution:** Causes the LLM to overlook key facts.
- **Noise Integration:** Injects irrelevant or conflicting assertions.

FACT

Retrieval recall growth does not equal answer quality growth. relevance filtering is required.

What is **metadata filtering**?

Applying hard SQL-like filters to search scopes before matching vector similarities:

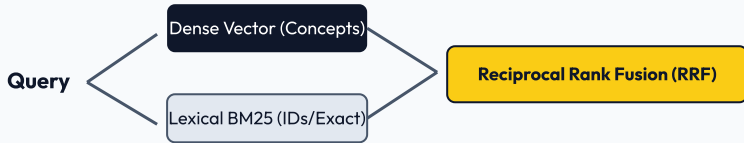


BENEFITS

Enforces strict access control, lowers search latency, and prevents database matching errors.

What is **hybrid search**?

Hybrid search combines semantic similarity with lexical keyword algorithms (BM25):



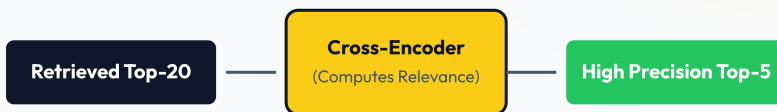
RULE OF THUMB

Vector search captures semantics; keyword search captures product codes, names, or error IDs accurately.

What is **reranking**?

Reranking runs a cross-encoder model to evaluate the logical similarity between query and retrieved document pairs.

Unlike fast approximate nearest neighbor search models, a reranker spends more compute resources to optimize precision:



WHY?

The vector search database provides speed and recall. The cross-encoder reranker ensures context relevance.

Why use **two-stage retrieval**?

A two-stage architecture separates the two key goals of information retrieval pipelines:

- **Stage 1 (Bi-Encoder):** Dense vector database query. Extremely fast, targets high ****recall**** rates to avoid missing facts.
- **Stage 2 (Cross-Encoder):** Compares candidates. Slower but highly accurate, targets ****precision**** to filter out noise.

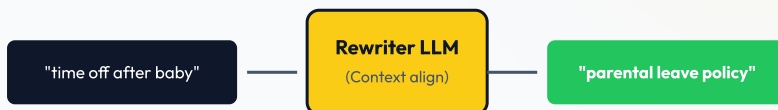
STANDARD PRACTICE

This hybrid combination optimizes both serving speed and prompt accuracy, lowering inference costs.

What is **query rewriting**?

Translating user queries into the terminology used by the indexing system.

If a user asks: "How much time off can I take after baby?", the document says "parental leave policy."



OBJECTIVE

Aligns user queries with indexing terminology to improve search recall.

What is **query expansion**?

Query expansion generates multiple search variations to capture synonyms and different phrasings.

For example, "remote work policy" expands into: "work from home guidelines", "telecommuting rules", and "hybrid schedules".

+ Pros:Increases recall rates.

- Cons:Adds search latency and vector database queries.

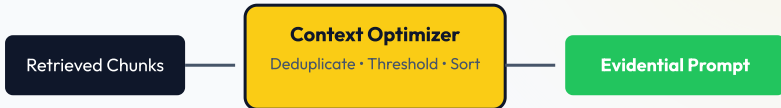
SYSTEM RULE

Only deploy expansion queries if search recall is low and latency budgets allow for parallel queries.



What is **context construction**?

Constructing the final prompt context from retrieved candidate chunks:



IMPORTANCE

Avoid simply concatenating text chunks. Deduplicate, sort by relevance, and structure layout tags for prompt clarity.



Why does **chunk ordering** matter?

Simply concatenating retrieved chunks randomly degrades generation quality.

Ordering strategies include:

- Placing most relevant evidence at the top or bottom boundaries of prompts to avoid "lost-in-the-middle" issues.
- Sorting chunks chronologically for logs or conversation streams.
- Grouping sections matching identical source documents.

GOAL

Shape input context blocks to make them easy for model reasoning paths to parse.



What should a **RAG prompt** hold?

A production-grade prompt template structures the input boundaries explicitly:

- **System Role:** Defines behavior limitations.
- **Grounding Rules:** Prevents out-of-context reasoning.
- **Retrieve Context Tags:** Clear XML markers (e.g. <context>).
- **Query block:** User question insertion.

EXAMPLE CONSTRAINT

"Answer using *only* the context below. If context does not support the answer, reply: 'Information not found.'"



How do citations help?

Citations map claims in generated answers back to matching database sources.

Example: "leave limit is 20 days [HR Policy, p. 4]".



WARNING

Citations build user trust, but engineers must validate that cited sources actually support the generated claims.



Debugging irrelevant chunks

If vector retrieval returns bad candidates, debug the search stack sequentially:

- Verify document parsing extracted clean text.
- Check if chunk sizes are too large or too small.
- Test if query semantics match embedding vocabulary.
- Implement metadata pre-filtering to narrow the search space.

FIRST ACTION

Ensure the target information is actually present in the source files before changing database search parameters.

Right context, **wrong answer**?

If document retrieval succeeds but output answers remain incorrect, debug the generation layers:

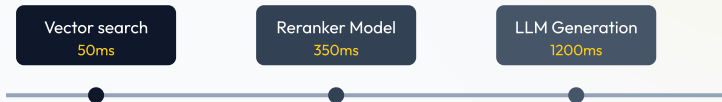
- The retrieved chunk lacks surrounding sentence context.
- Conflicting chunks confuse prompt logic.
- Prompt instructions permit excessive model guessing.

DEBUGGING FOCUS

Isolate your system. Test retrieval quality (recall) separately from generation quality (faithfulness).

How to debug high latency?

Break query pipeline duration down into separate execution phases to isolate bottlenecks:



RULE

Isolate bottlenecks first. Optimize vector index speed, restrict reranker pools, and stream output tokens.

How to reduce RAG costs?

Context input tokens represent the largest hidden fee in production systems. Optimize using these methods:

- > **Rerank and filter:** Reduce Top-K context counts.
- > **Context Compression:** Summarize or strip out filler words from retrieved text blocks.
- > **Semantic Caching:** Reuse answers for repeating queries.

OPTIMIZATION FLOW

Retrieve broadly → Rerank → Compress → Send only top evidence → LLM.

How to evaluate **RAG**?

Do not test only final answers. Benchmark three separate aspects to isolate pipeline performance issues:

- 🔍 **Retrieval accuracy:** Precision, recall, and context coverage.
- 🏰 **Generation quality:** Correctness, complete reasoning, and style.
- 🛡 **Faithfulness:** Verifying grounding constraints.

STANDARD FRAMEWORK

Evaluating each layer independently isolates failures, ensuring prompt edits do not break search logic.

What is retrieval relevance?

Retrieval relevance calculates the ratio of useful facts contained in retrieved chunks relative to the user query.



RELEVANCE METRIC

A pipeline that regularly returns noise chunks (like 3 and 4) suffers from retrieval precision issues.

What is **faithfulness** in RAG?

Faithfulness measures if the model's generated answer is supported *exclusively* by the retrieved context chunks.

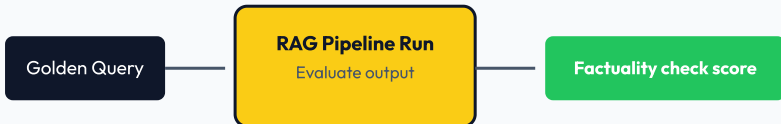
If context states: "vacation is 20 days", and the answer claims: "vacation is 20 days and carries over for 10" without carry-over data in the source chunks, the answer is unfaithful.

CRUCIAL DISTINCTION

An answer can be factually true in the real world but still unfaithful if the retrieved document chunks do not support it.

What is a **golden dataset**?

A golden dataset is a curated regression testing suite of representative user query samples, document reference keys, and expected outputs.



METHODOLOGY

Run golden dataset evaluations whenever you update chunk sizes, database embeddings, or prompts to verify system changes.

When is hybrid search **better**?

Deploy hybrid search when user queries contain mixed semantic concepts and keyword identifiers:

- ✓ **Concept matching:** "how do I request vacation leaves?" (Dense vector semantic match).
- ✓ **Lexical matching:** "error code AUTH-403" (Lexical BM25 exact match).

SELECTION RULE

Do not guess. Analyze production query logs — if search queries contain alphanumeric codes or exact SKU names, implement hybrid search.

When should you **NOT** use RAG?

RAG adds latency and ingestion costs. Avoid semantic retrieval in these cases:

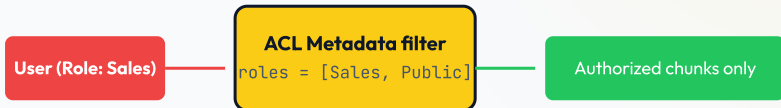
- Pure text transformations (e.g. "format this text block").
- The query requires real-time transaction updates (e.g. "my current credit balance?") — use structured APIs instead.
- The model already holds sufficient parametric knowledge.

INTERVIEW INSIGHT

Use retrieval augmented pipelines strictly to ground queries with external evidence — not because RAG is fashionable.

Multi-tenant RAG security

Never rely on system prompt instructions to secure document access permissions. Secure your system at the database layer:



PRINCIPLE

The vector search database must block unauthorized chunks before they reach prompt structures.

Design a production **RAG**

A production-ready RAG architecture requires separating search from generation layers:

- ✓ **Ingestion:** Document parsing → chunking → embedding → vector database.
- ✓ **Search:** Query rewrite → hybrid search → metadata filters → cross-encoder reranker.
- ✓ **Generation:** Context builders → prompt templates → LLM inference → response verification.

TAKEAWAY

A production RAG application is not just a vector index coupled with an API call. It is a complete information retrieval system.

Your RAG Answer is **Wrong?**

Follow this debugging hierarchy to locate system errors:

1. Check source files

Does the correct information exist in the database?

2. Check parsing

Was document structure extracted cleanly?

3. Check chunking

Was semantic continuity maintained?

4. Check embedding

Did vector space metrics yield clean matches?

5. Check retrieval

Did correct facts make the Top-K list?

6. Check rerank

Did cross-encoders promote target evidence?



DEBUGGING AXIOM

Do not debug the LLM first. Trace upstream document extraction, database vector similarity, and context filters.

RAG Interview **Answers**

Improving Retrieval

Explain chunking, vector embeddings, metadata filters, hybrid BM25 searches, and cross-encoder rerankers.

Reducing Hallucinations

Talk about factual grounding constraints, citation mappings, faithfulness metrics, and fallback handling.

Lowering Latency

Mention approximate nearest neighbor index speed, reranking threshold caps, semantic caching, and token output streaming.

Optimizing Cost

Detail document compression, input context token limits, semantic search reuse, and cheap classification models.

System Evaluation Frameworks

Benchmark context precision and recall, faithfulness assertions, and direct answer relevance ratios.

✔ Use these pillars to structure RAG architecture answers.

Don't Say: "I built a RAG app."

Be ready to answer the engineering decisions:

- **Why** this chunk size and overlap ratio?
- **Why** this database embedding model and index?
- **Why** reranking, hybrid search, or query rewriting?
- **How** do you evaluate retrieval quality and secure tenants?

"Explain why your RAG system works — and how you would fix it when it doesn't."

✓ Day 2 Complete

🔖 Save this series

📅 **Next:** 30–40 Chunking Interview Questions